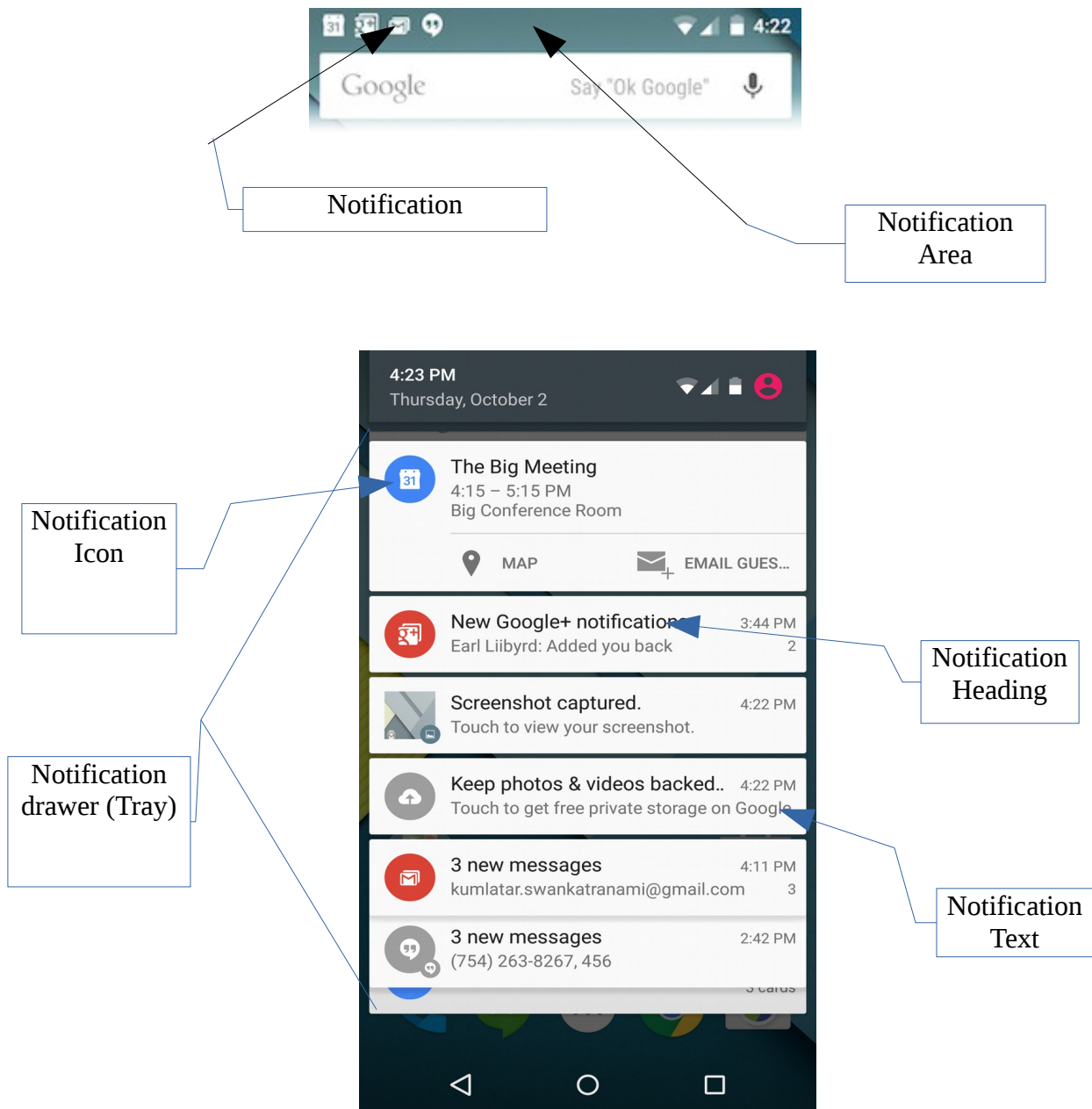


Introduction to Notification

A notification is a message you can display to the user outside of your application's normal UI. When you tell the system to issue a notification, it first appears as an icon in the **notification area**. To see the details of the notification, the user opens the **notification drawer**. Both the notification area and the notification drawer are system-controlled areas that the user can view at any time.



Creating notification

To create notification we have to specify UI information and action for notification in a **NotificationCompat.Builder** object.

Once the UI is set we can call **NotificationCompat.Builder.build()** which returns a **Notification** object containing your specification. To issue notification you can pass the **Notification** Object to the system by calling **NotificationManager.notify()**

A Notification Object must be set with following content.

- A small icon, set by [setSmallIcon\(\)](#).
- A title set by [setContentTitle\(\)](#)
- Text set by [setContentText\(\)](#)

You can set other content from “Public Methods” of **NotificationCompat.Builder**

Notification actions

Although they're optional, you should add at least one action to your notification. An action allows users to go directly from the notification to an [Activity](#) in your application, where they can look at one or more events or do further work.

A notification can provide multiple actions. You should always define the action that's triggered when the user clicks the notification; usually this action opens an [Activity](#) in your application. You can also add buttons to the notification that perform additional actions such as snoozing an alarm or responding immediately to a text message; this feature is available as of Android 4.1. If you use additional action buttons, you must also make their functionality available in an [Activity](#) in your app; see the section [Handling compatibility](#) for more details.

Inside a [Notification](#), the action itself is defined by a [PendingIntent](#) containing an [Intent](#) that starts an [Activity](#) in your application. To associate the [PendingIntent](#) with a gesture, call the appropriate method of [NotificationCompat.Builder](#). For example, if you want to start [Activity](#) when the user clicks the notification text in the notification drawer, you add the [PendingIntent](#) by calling [setContentIntent\(\)](#).

Starting an [Activity](#) when the user clicks the notification is the most common action scenario. You can also start an [Activity](#) when the user dismisses a notification. In Android 4.1 and later, you can start an [Activity](#) from an action button.

Sample Project Notification

Pre-requisites steps:

- 1) Create New Project with empty template
- 2) Create button in the activity and assign **onClick** event (Named: showNote)
- 3) Create xml layout for new activity (noteactivity.xml) with some content
- 4) create one more activity (ShowNote.java), We require that and want to open when the notification is pressed. Load noteactivity.xml in onCreate lifecycle of Java class
- 5) Modify the manifest file as required. In our file we can just increase following text

```
<activity android:name=".ShowNote" >
    <intent-filter>
        <action android:name="second_filter" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

- 6) Create a function for a click event. This should generate the notification with action direction.

```
public void showNote (View v){
    NotificationCompat.Builder builder = new NotificationCompat.Builder(this);
    builder.setSmallIcon(R.drawable.clogo);
    builder.setContentTitle("MCS Notification");
    builder.setContentText("This will open other activity ...");
    Intent intent = new Intent(this, ShowNote.class);
    TaskStackBuilder stackBuilder = TaskStackBuilder.create(this);
    stackBuilder.addParentStack(ShowNote.class);
    stackBuilder.addNextIntent(intent);
    PendingIntent pendingIntent = stackBuilder.getPendingIntent(0,
PendingIntent.FLAG_UPDATE_CURRENT);
    builder.setContentIntent(pendingIntent);
    builder.setAutoCancel(true);
    NotificationManager NM = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
    NM.notify(0, builder.build());
}
```

BroadcastReceiver Class

BroadcastReceiver Class is a base-class for the code that will receive the intent sent by `sendBroadcast()`.

If you don't need to send broadcasts across applications, consider using this class with [LocalBroadcastManager](#) instead of the more general facilities described below. This will give you a much more efficient implementation (no cross-process communication needed) and allow you to avoid thinking about any security issues related to other applications being able to receive or send your broadcasts.

You can either dynamically register an instance of this class with `Context.registerReceiver()` or statically publish an implementation through the `<receiver>` tag in your `AndroidManifest.xml`.

Note: If registering a receiver in your `Activity.onResume()` implementation, you should unregister it in `Activity.onPause()`. (You won't receive intents when paused, and this will cut down on unnecessary system overhead). Do not unregister in `Activity.onSaveInstanceState()`, because this won't be called if the user moves back in the history stack.

There are two major classes of broadcasts that can be received:

- **Normal broadcasts** (sent with `Context.sendBroadcast`) are completely asynchronous. All receivers of the broadcast are run in an undefined order, often at the same time. This is more efficient, but means that receivers cannot use the result or abort APIs included here.
- **Ordered broadcasts** (sent with `Context.sendOrderedBroadcast`) are delivered to one receiver at a time. As each receiver executes in turn, it can propagate a result to the next receiver, or it can completely abort the broadcast so that it won't be passed to other receivers. The order receivers run in can be controlled with the `android:priority` attribute of the matching intent-filter; receivers with the same priority will be run in an arbitrary order.

Even in the case of normal broadcasts, the system may in some situations revert to delivering the broadcast one receiver at a time. In particular, for receivers that may require the creation of a process, only one will be run at a time to avoid overloading the system with new processes. In this situation, however, the non-ordered semantics hold: these receivers still cannot return results or abort their broadcast.

Note that, although the Intent class is used for sending and receiving these broadcasts, the Intent broadcast mechanism here is completely separate from Intents that are used to start Activities with `Context.startActivity()`. There is no way for a BroadcastReceiver to see or capture Intents used with `startActivity()`; likewise, when you broadcast an Intent, you will never find or start an Activity. These two operations are semantically very different: starting an Activity with an Intent is a foreground operation that modifies what the user is currently interacting with; broadcasting an Intent is a background operation that the user is not normally aware of.

Processes and Threads

When an application component starts and the application does not have any other components running, the Android system starts a new Linux process for the application with a single thread of execution. By default, all components of the same application run in the same process and thread (called the "main" thread). If an application component starts and there already exists a process for that application (because another component from the application exists), then the component is started within that process and uses the same thread of execution. However, you can arrange for different components in your application to run in separate processes, and you can create additional threads for any process.

Processes

By default, all components of the same application run in the same process and most applications should not change this. However, if you find that you need to control which process a certain component belongs to, you can do so in the manifest file.

The manifest entry for each type of component element—`<activity>`, `<service>`, `<receiver>`, and `<provider>`—supports an `android:process` attribute that can specify a process in which that component should run. You can set this attribute so that each component runs in its own process or so that some components share a process while others do not. You can also set `android:process` so that components of different applications run in the same process—provided that the applications share the same Linux user ID and are signed with the same certificates.

The `<application>` element also supports an `android:process` attribute, to set a default value that applies to all components.

Android might decide to shut down a process at some point, when memory is low and required by other processes that are more immediately serving the user. Application components running in the process that's killed are consequently destroyed. A process is started again for those components when there's again work for them to do.

When deciding which processes to kill, the Android system weighs their relative importance to the user. For example, it more readily shuts down a process hosting activities that are no longer visible on screen, compared to a process hosting visible activities. The decision whether to terminate a process, therefore, depends on the state of the components running in that process. The rules used to decide which processes to terminate is discussed below.

Process Lifecycle

The Android system tries to maintain an application process for as long as possible, but eventually needs to remove old processes to reclaim memory for new or more important processes. To determine which processes to keep and which to kill, the system places each process into an "importance hierarchy" based on the components running in the process and the state of those components. Processes with the lowest importance are eliminated first, then those with the next lowest importance, and so on, as necessary to recover system resources.

There are five levels in the importance hierarchy. The following list presents the different types of processes in order of importance (the first process is *most important* and is *killed last*):

1.Foreground process

A process that is required for what the user is currently doing. A process is considered to be in the foreground if any of the following conditions are true:

- It hosts an [Activity](#) that the user is interacting with (the [Activity's onResume\(\)](#) method has been called).
- It hosts a [Service](#) that's bound to the activity that the user is interacting with.
- It hosts a [Service](#) that's running "in the foreground"—the service has called [startForeground\(\)](#).
- It hosts a [Service](#) that's executing one of its lifecycle callbacks ([onCreate\(\)](#), [onStart\(\)](#), or [onDestroy\(\)](#)).
- It hosts a [BroadcastReceiver](#) that's executing its [onReceive\(\)](#) method.

Generally, only a few foreground processes exist at any given time. They are killed only as a last resort—if memory is so low that they cannot all continue to run. Generally, at that point, the device has reached a memory paging state, so killing some foreground processes is required to keep the user interface responsive.

2.Visible process

A process that doesn't have any foreground components, but still can affect what the user sees on screen. A process is considered to be visible if either of the following conditions are true:

- It hosts an [Activity](#) that is not in the foreground, but is still visible to the user (its [onPause\(\)](#) method has been called). This might occur, for example, if the foreground activity started a dialog, which allows the previous activity to be seen behind it.
- It hosts a [Service](#) that's bound to a visible (or foreground) activity.

A visible process is considered extremely important and will not be killed unless doing so is required to keep all foreground processes running.

3.Service process

A process that is running a service that has been started with the [startService\(\)](#) method and does not fall into either of the two higher categories. Although service processes are not directly tied to anything the user sees, they are generally doing things that the user cares about (such as playing music in the background or downloading data on the network), so the system keeps them running unless there's not enough memory to retain them along with all foreground and visible processes.

4.Background process

A process holding an activity that's not currently visible to the user (the activity's [onStop\(\)](#) method has been called). These processes have no direct impact on the user experience, and the system can kill them at any time to reclaim memory for a foreground, visible, or service process. Usually there are many background processes running, so they are kept in an LRU (least recently used) list to ensure that the process with the activity that was most recently seen by the user is

the last to be killed. If an activity implements its lifecycle methods correctly, and saves its current state, killing its process will not have a visible effect on the user experience, because when the user navigates back to the activity, the activity restores all of its visible state. See the [Activities](#) document for information about saving and restoring state.

5.Empty process

A process that doesn't hold any active application components. The only reason to keep this kind of process alive is for caching purposes, to improve startup time the next time a component needs to run in it. The system often kills these processes in order to balance overall system resources between process caches and the underlying kernel caches.

Android ranks a process at the highest level it can, based upon the importance of the components currently active in the process. For example, if a process hosts a service and a visible activity, the process is ranked as a visible process, not a service process.

In addition, a process's ranking might be increased because other processes are dependent on it—a process that is serving another process can never be ranked lower than the process it is serving. For example, if a content provider in process A is serving a client in process B, or if a service in process A is bound to a component in process B, process A is always considered at least as important as process B.

Threads

When an application is launched, the system creates a thread of execution for the application, called "main." This thread is very important because it is in charge of dispatching events to the appropriate user interface widgets, including drawing events. It is also the thread in which your application interacts with components from the Android UI toolkit (components from the [android.widget](#) and [android.view](#) packages). As such, the main thread is also sometimes called the UI thread.

The system does *not* create a separate thread for each instance of a component. All components that run in the same process are instantiated in the UI thread, and system calls to each component are dispatched from that thread. Consequently, methods that respond to system callbacks (such as [onKeyDown\(\)](#) to report user actions or a lifecycle callback method) always run in the UI thread of the process.

For instance, when the user touches a button on the screen, your app's UI thread dispatches the touch event to the widget, which in turn sets its pressed state and posts an invalidate request to the event queue. The UI thread dequeues the request and notifies the widget that it should redraw itself.

When your app performs intensive work in response to user interaction, this single thread model can yield poor performance unless you implement your application properly. Specifically, if everything is happening in the UI thread, performing long operations such as network access or database queries will block the whole UI. When the thread is blocked, no events can be dispatched, including drawing events. From the user's perspective, the application appears to hang. Even worse, if the UI

thread is blocked for more than a few seconds (about 5 seconds currently) the user is presented with the infamous "[application not responding](#)" (ANR) dialog. The user might then decide to quit your application and uninstall it if they are unhappy.

Additionally, the Android UI toolkit is *not* thread-safe. So, you must not manipulate your UI from a worker thread—you must do all manipulation to your user interface from the UI thread. Thus, there are simply two rules to Android's single thread model:

1. Do not block the UI thread
2. Do not access the Android UI toolkit from outside the UI thread

Worker threads

Because of the single thread model described above, it's vital to the responsiveness of your application's UI that you do not block the UI thread. If you have operations to perform that are not instantaneous, you should make sure to do them in separate threads ("background" or "worker" threads).

For example, below is some code for a click listener that downloads an image from a separate thread and displays it in an [ImageView](#):

```
public void onClick(View v) {
    new Thread(new Runnable() {
        public void run() {
            Bitmap b = loadImageFromNetwork("http://example.com/image.png");
            mImageView.setImageBitmap(b);
        }
    }).start();
}
```

At first, this seems to work fine, because it creates a new thread to handle the network operation. However, it violates the second rule of the single-threaded model: *do not access the Android UI toolkit from outside the UI thread*—this sample modifies the [ImageView](#) from the worker thread instead of the UI thread. This can result in undefined and unexpected behavior, which can be difficult and time-consuming to track down.

To fix this problem, Android offers several ways to access the UI thread from other threads. Here is a list of methods that can help:

- [Activity.runOnUiThread\(Runnable\)](#)
- [View.post\(Runnable\)](#)
- [View.postDelayed\(Runnable, long\)](#)

For example, you can fix the above code by using the [View.post\(Runnable\)](#) method:

```
public void onClick(View v) {
    new Thread(new Runnable() {
        public void run() {
            final Bitmap bitmap =
                loadImageFromNetwork("http://example.com/image.png");
```



```

        mImageView.post(new Runnable() {
            public void run() {
                mImageView.setImageBitmap(bitmap);
            }
        });
    }
}).start();
}

```

Now this implementation is thread-safe: the network operation is done from a separate thread while the `ImageView` is manipulated from the UI thread.

However, as the complexity of the operation grows, this kind of code can get complicated and difficult to maintain. To handle more complex interactions with a worker thread, you might consider using a `Handler` in your worker thread, to process messages delivered from the UI thread. Perhaps the best solution, though, is to extend the `AsyncTask` class, which simplifies the execution of worker thread tasks that need to interact with the UI.

AsyncTask

`AsyncTask` enables proper and easy use of the UI thread. This class allows to perform background operations and publish results on the UI thread without having to manipulate threads and/or handlers.

`AsyncTask` is designed to be a helper class around `Thread` and `Handler` and does not constitute a generic threading framework. `AsyncTasks` should ideally be used for short operations (a few seconds at the most.) If you need to keep threads running for long periods of time, it is highly recommended you use the various APIs provided by the `java.util.concurrent` package such as `Executor`, `ThreadPoolExecutor` and `FutureTask`.

An asynchronous task is defined by a computation that runs on a background thread and whose result is published on the UI thread. An asynchronous task is defined by 3 generic types, called `Params`, `Progress` and `Result`, and 4 steps, called `onPreExecute`, `doInBackground`, `onProgressUpdate` and `onPostExecute`.

Usage

`AsyncTask` must be subclassed to be used. The subclass will override at least one method (`doInBackground(Params...)`), and most often will override a second one (`onPostExecute(Result)`).

Here is an example of subclassing:

```

private class DownloadFilesTask extends AsyncTask<URL, Integer, Long> {
    protected Long doInBackground(URL... urls) {
        int count = urls.length;
        long totalSize = 0;
        for (int i = 0; i < count; i++) {
            totalSize += Downloader.downloadFile(urls[i]);
            publishProgress((int) ((i / (float) count) * 100));
            // Escape early if cancel() is called
            if (isCancelled()) break;
        }
        return totalSize;
    }
    protected void onProgressUpdate(Integer... progress) {

```

```

        setProgressPercent(progress[0]);
    }
    protected void onPostExecute(Long result) {
        showDialog("Downloaded " + result + " bytes");
    }
}

```

Once created, a task is executed very simply:

```
new DownloadFilesTask().execute(url1, url2, url3);
```

AsyncTask's generic types

The three types used by an asynchronous task are the following:

1. **Params**, the type of the parameters sent to the task upon execution.
2. **Progress**, the type of the progress units published during the background computation.
3. **Result**, the type of the result of the background computation.

Not all types are always used by an asynchronous task. To mark a type as unused, simply use the type **Void**:

```
private class MyTask extends AsyncTask<Void, Void, Void> { ... }
```

The 4 steps

When an asynchronous task is executed, the task goes through 4 steps:

1. **onPreExecute()**, invoked on the UI thread before the task is executed. This step is normally used to setup the task, for instance by showing a progress bar in the user interface.
2. **doInBackground(Params...)**, invoked on the background thread immediately after **onPreExecute()** finishes executing. This step is used to perform background computation that can take a long time. The parameters of the asynchronous task are passed to this step. The result of the computation must be returned by this step and will be passed back to the last step. This step can also use **publishProgress(Progress...)** to publish one or more units of progress. These values are published on the UI thread, in the **onProgressUpdate(Progress...)** step.
3. **onProgressUpdate(Progress...)**, invoked on the UI thread after a call to **publishProgress(Progress...)**. The timing of the execution is undefined. This method is used to display any form of progress in the user interface while the background computation is still executing. For instance, it can be used to animate a progress bar or show logs in a text field.
4. **onPostExecute(Result)**, invoked on the UI thread after the background computation finishes. The result of the background computation is passed to this step as a parameter.

Cancelling a task

A task can be cancelled at any time by invoking `cancel(boolean)`. Invoking this method will cause subsequent calls to `isCancelled()` to return true. After invoking this method, `onCancelled(Object)`, instead of `onPostExecute(Object)` will be invoked after `doInBackground(Object[])` returns. To ensure that a task is cancelled as quickly as possible, you should always check the return value of `isCancelled()` periodically from `doInBackground(Object[])`, if possible (inside a loop for instance.)

Threading rules

There are a few threading rules that must be followed for this class to work properly:

- The AsyncTask class must be loaded on the UI thread. This is done automatically as of `JELLY_BEAN`.
- The task instance must be created on the UI thread.
- `execute(Params...)` must be invoked on the UI thread.
- Do not call `onPreExecute()`, `onPostExecute(Result)`, `doInBackground(Params...)`, `onProgressUpdate(Progress...)` manually.
- The task can be executed only once (an exception will be thrown if a second execution is attempted.)

Memory observability

AsyncTask guarantees that all callback calls are synchronized in such a way that the following operations are safe without explicit synchronizations.

- Set member fields in the constructor or `onPreExecute()`, and refer to them in `doInBackground(Params...)`.
- Set member fields in `doInBackground(Params...)`, and refer to them in `onProgressUpdate(Progress...)` and `onPostExecute(Result)`.

Order of execution

When first introduced, AsyncTasks were executed serially on a single background thread. Starting with `DONUT`, this was changed to a pool of threads allowing multiple tasks to operate in parallel. Starting with `HONEYCOMB`, tasks are executed on a single thread to avoid common application errors caused by parallel execution.

If you truly want parallel execution, you can invoke `executeOnExecutor(java.util.concurrent.Executor, Object[])` with `THREAD_POOL_EXECUTOR`

Handlers

A Handler allows you to send and process [Message](#) and [Runnable](#) objects associated with a thread's [MessageQueue](#). Each Handler instance is associated with a single thread and that thread's message queue. When you create a new Handler, it is bound to the thread / message queue of the thread that is creating it -- from that point on, it will deliver messages and runnables to that message queue and execute them as they come out of the message queue.

There are two main uses for a Handler: (1) to schedule messages and runnables to be executed at some point in the future; and (2) to enqueue an action to be performed on a different thread than your own.

Scheduling messages is accomplished with the [post\(Runnable\)](#), [postAtTime\(Runnable, long\)](#), [postDelayed\(Runnable, long\)](#), [sendEmptyMessage\(int\)](#), [sendMessage\(Message\)](#), [sendMessageAtTime\(Message, long\)](#), and [sendMessageDelayed\(Message, long\)](#) methods. The *post* versions allow you to enqueue [Runnable](#) objects to be called by the message queue when they are received; the *sendMessage* versions allow you to enqueue a [Message](#) object containing a bundle of data that will be processed by the Handler's [handleMessage\(Message\)](#) method (requiring that you implement a subclass of Handler).

When posting or sending to a Handler, you can either allow the item to be processed as soon as the message queue is ready to do so, or specify a delay before it gets processed or absolute time for it to be processed. The latter two allow you to implement timeouts, ticks, and other timing-based behavior.

When a process is created for your application, its main thread is dedicated to running a message queue that takes care of managing the top-level application objects (activities, broadcast receivers, etc) and any windows they create. You can create your own threads, and communicate back with the main application thread through a Handler. This is done by calling the same *post* or *sendMessage* methods as before, but from your new thread. The given [Runnable](#) or [Message](#) will then be scheduled in the Handler's message queue and processed when appropriate.

Network

Connection of two or more devices through any medium is a network. Android can communicate with different network services such as Mobile Data Network, Bluetooth, NFC, WiFi, USB etc.

Creating Network application:

Purpose of this application will be to download some text from any given url.

We'll be using HTTP client to download the feed. In android we can get **HttpURLConnection** client which will allow us to work with HTTPS, Streaming uploads and downloads, configurable timeouts, IPv6 and connection pooling.

Step 1>

We need permission to use INTERNET and NETWORK_STATE services. We can achieve it by adding below lines in our manifests file

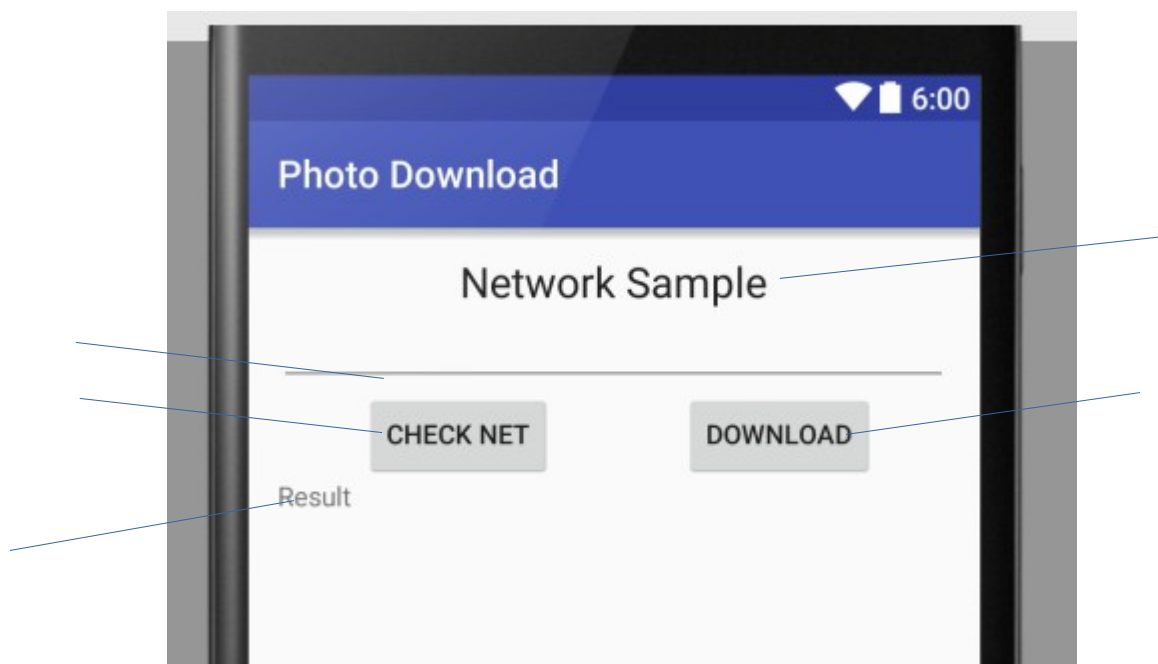
```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

android.permission.INTERNET → This will allow application to open network socket.

android.permission.ACCESS_NETWORK_STATE → This allows application to access information about networks.

Step 2>

Make the Layout ready with following components



Step 3)

Create Button to check the network connectivity

//// Checking Network connection through button

Process 1) Use connectivity manager to instantiate connectivity service.

Process 2) Use Network info object to receive the status of the connection

Note: The result will come in boolean

Process 3) If Network info is true, take Toast message and inform user **Positive**. If not take toast message and inform Negative>

```
public void checkBtn(View v){
    ConnectivityManager connMgr = (ConnectivityManager)
        getSystemService(Context.CONNECTIVITY_SERVICE);
    NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();
    if (networkInfo != null && networkInfo.isConnected()) {
        Toast.makeText(getBaseContext(), "Network is connected",
            Toast.LENGTH_LONG).show();
    } else {
        Toast.makeText(getBaseContext(), "!!! Not connected",
            Toast.LENGTH_LONG).show();
    }
}
```

Step 4) Create the button to download the content.

The content with entire project is attached in .zip file.